

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1717

Annexure A

CASE STUDY

Name: Hemalaksbmi H

Reg. No.: 192571049

Case Study 1 — Smart Medical Diagnosis System

1. Introduction

A hospital is developing a rule-based logical agent to assist in preliminary medical diagnosis. The agent uses propositional logic and inference mechanisms such as Forward Chaining and Backward Chaining to derive possible diseases from patient symptoms. This report presents the logical representation of rules, inference steps, and goal-directed reasoning for Patient A.

2. Knowledge Base Description

The knowledge base contains three diagnostic rules and factual information about Patient A.

2.1 Defined Propositional Symbols

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Patient has Possible Flu

Symbol	Meaning
Mea	Patient has Possible Measles
Pneu	Patient has Possible Pneumonia

2.2 Rules in Propositional Logic

1. $F \wedge C \rightarrow \text{Flu}$
2. $F \wedge R \rightarrow \text{Mea}$
3. $C \wedge B \rightarrow \text{Pneu}$

2.3 Facts about Patient A

- $F = \text{True}$
- $C = \text{True}$
- $B = \text{True}$

3. Forward Chaining Inference

Forward Chaining is a data-driven reasoning method. We start from known facts and apply rules whose premises are satisfied.

Step-by-Step Inference

Step 1: Check Rule 1

Rule: $F \wedge C \rightarrow \text{Flu}$ Facts: $F = \text{True}$, $C = \text{True}$ Premise satisfied $\rightarrow$ Rule fires

Derived Fact: $\text{Flu} = \text{True}$

Step 2: Check Rule 2

Rule: $F \wedge R \rightarrow \text{Mea}$ Facts: $F = \text{True}$, $R = \text{False}$ (not given) Premise not satisfied $\rightarrow$ Rule does not fire

No new fact derived

Step 3: Check Rule 3

Rule: $C \wedge B \rightarrow \text{Pneu}$ Facts: $C = \text{True}$, $B = \text{True}$ Premise satisfied $\rightarrow$ Rule fires

Derived Fact: $\text{Pneu} = \text{True}$

3.1 Final Forward Chaining Conclusions

Diagnosis	Result
Possible Flu	True
Possible Measles	Not derivable
Possible Pneumonia	True

Thus, Patient A is diagnosed with:

- Possible Flu
- Possible Pneumonia

4. Backward Chaining for Goal: "Patient A has Pneumonia"

Backward Chaining is goal-driven reasoning. We start from the goal and reduce it into sub-goals until we reach known facts.

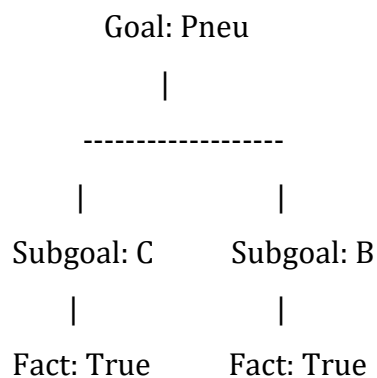
Goal:

Pneu

Rule Supporting Goal:

Rule 3: $C \wedge B \rightarrow \text{Pneu}$

Goal Reduction Tree



Explanation

- To prove Pneu, the agent checks Rule 3.
- Rule 3 requires C and B.
- Both C and B are present in the facts.
- Therefore, the goal Pneu is successfully proven.

Backward Chaining Result

Confirmed: Patient A has Pneumonia

5. Conclusion

The Smart Medical Diagnosis System successfully uses propositional logic and inference mechanisms to diagnose Patient A.

Forward Chaining identifies Possible Flu and Possible Pneumonia. Backward Chaining confirms the goal Pneumonia by reducing it to known symptoms. This demonstrates how rule-based logical agents can support preliminary medical decision-making.

Case Study 2 — Autonomous Traffic Management Agent

1. Introduction

Modern smart cities rely on intelligent traffic management systems to ensure safe and efficient movement of vehicles. Emergency vehicles such as ambulances require immediate clearance and priority routing. This case study analyzes a First-Order Logic (FOL)-based knowledge base designed to manage emergency vehicle routing. The report demonstrates how unification and resolution are applied to derive logical conclusions and evaluates whether the current knowledge base is sufficient for handling multiple simultaneous emergency vehicles.

2. Knowledge Base Description

The knowledge base consists of three rules and four facts:

Rules

1. **Rule 1:** $\forall x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$
2. **Rule 2:** $\forall x: \text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$
3. **Rule 3:** $\forall x \forall y: \text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$

Facts

- $\text{Vehicle}(\text{Ambulance})$
- $\text{EmergencyType}(\text{Ambulance})$
- $\text{Vehicle}(\text{CarA})$
- $\text{Behind}(\text{CarA}, \text{Ambulance})$

3. Part (a): Unification of Rule 1 with Facts

Unification is used to match the variables in Rule 1 with the constants in the facts.

Step-by-step Unification

- Match $\text{Vehicle}(x)$ with $\text{Vehicle}(\text{Ambulance}) \rightarrow$ Substitution $\theta_1 = \{x / \text{Ambulance}\}$
- Apply θ_1 to $\text{EmergencyType}(x) \rightarrow \text{EmergencyType}(\text{Ambulance})$, which matches the fact.

Most General Unifier (MGU)

$$\theta = \{x / \text{Ambulance}\}$$

Result

Using θ , Rule 1 derives:

$$\text{ClearPath}(\text{Ambulance})$$

4. Part (b): Resolution Proof for $\text{Proceed}(\text{CarA})$

Resolution is used to prove that the knowledge base logically entails:

$$\text{Proceed}(\text{CarA})$$

4.1 Conversion of Rules to CNF

Rule CNF Form

Rule 1 $\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)$

Rule 2a $\neg \text{ClearPath}(x) \vee \text{GreenSignal}(x)$

Rule 2b $\neg \text{ClearPath}(x) \vee \neg \text{RedSignal}(x)$

Rule 3 $\neg \text{Vehicle}(x) \vee \neg \text{Behind}(x, y) \vee \neg \text{GreenSignal}(y) \vee \text{Proceed}(x)$

Facts remain as unit clauses.

Goal for resolution:

$$\neg \text{Proceed}(\text{CarA})$$

4.2 Resolution Steps

Step 1: Derive ClearPath(Ambulance)

Using Rule 1 + Vehicle(Ambulance) + EmergencyType(Ambulance):

$$\text{ClearPath}(\text{Ambulance})$$

Step 2: Derive GreenSignal(Ambulance)

Using Rule 2a + ClearPath(Ambulance):

$$\text{GreenSignal}(\text{Ambulance})$$

Step 3: Derive Proceed(CarA)

Using Rule 3 + Vehicle(CarA) + Behind(CarA, Ambulance) + GreenSignal(Ambulance):

$$\text{Proceed}(\text{CarA})$$

Step 4: Resolution with Negated Goal

Resolve:

- $\text{Proceed}(\text{CarA})$
- $\neg \text{Proceed}(\text{CarA})$

Result:

(Empty Clause)

Conclusion of Part (b)

The empty clause confirms that the knowledge base **logically proves**:

$$\text{Proceed}(\text{CarA})$$

5. Part (c): Evaluation of Knowledge Base for Multiple Emergency Vehicles

Current Limitations

The existing knowledge base assumes:

- Only **one** emergency vehicle at a time.
- No intersection conflict handling.
- No priority rules.
- No mutual exclusion of signals.

If two emergency vehicles (e.g., Ambulance1 and Ambulance2) approach the same intersection, the current rules would incorrectly assign **GreenSignal** to both, creating a dangerous situation.

Required Enhancements

To safely manage multiple emergency vehicles, the following additions are necessary:

1. Intersection Conflict Detection

$$EmergencyType(v_1) \wedge EmergencyType(v_2) \wedge OnRoute(v_1, i) \wedge OnRoute(v_2, i) \\ \rightarrow Conflict(i, v_1, v_2)$$

2. Priority Assignment Rule

$$Conflict(i, v_1, v_2) \wedge HigherPriority(v_1, v_2) \rightarrow GrantRightOfWay(v_1, i)$$

3. Signal Coordination Rule

$$GrantRightOfWay(v, i) \rightarrow GreenSignal(v) \wedge \forall u (u \neq v \rightarrow RedSignal(u))$$

4. Safety Constraint

$$\neg (GreenSignal(v_1) \wedge GreenSignal(v_2))$$

Conclusion of Part (c)

The current knowledge base is **not sufficient** for handling multiple emergency vehicles. Additional rules for conflict detection, priority management, and coordinated signal control are essential for safe operation.

6. Final Conclusion

This report demonstrates how unification and resolution techniques allow a smart traffic management agent to infer correct actions for emergency vehicle routing. While the existing knowledge base successfully handles single emergency vehicle scenarios, it lacks the necessary logical structure to manage multiple simultaneous emergencies. Enhancing the knowledge base with conflict, priority, and safety rules is crucial for real-world deployment in smart city environments.

Case Study 3 — Agricultural Expert Reasoning System

1. Introduction

Agricultural decision-making increasingly relies on logical agents to provide precise, rule-based recommendations. In this case study, an agricultural advisory system uses propositional logic to reason about irrigation decisions and crop risk for wheat. This report converts the given knowledge base into Conjunctive Normal Form (CNF), applies resolution refutation to prove a target theorem, and analyzes how conclusions change when a key fact is removed.

2. Knowledge base description

2.1 Propositional rules

- **P1:** $SoilDry \rightarrow IrrigationNeeded$
- **P2:** $IrrigationNeeded \wedge CropWheat \rightarrow ApplyDripMethod$
- **P3:** $\neg ApplyDripMethod \wedge CropWheat \rightarrow CropAtRisk$

2.2 Facts

- $SoilDry = True$
- $CropWheat = True$

3. Part (a): CNF conversion of P1, P2, P3 and facts

We follow the standard steps:

1. Eliminate implication
2. Move negation inward (if needed)
3. Distribute $\vee$ over $\wedge$ to get CNF

3.1 CNF for P1

Original:

$$SoilDry \rightarrow IrrigationNeeded$$

Step 1: Eliminate implication

$$\neg SoilDry \vee IrrigationNeeded$$

No further negation or distribution needed.

CNF (P1):

$$(\neg SoilDry \vee IrrigationNeeded)$$

3.2 CNF for P2

Original:

$$IrrigationNeeded \wedge CropWheat \rightarrow ApplyDripMethod$$

Step 1: Eliminate implication

$$\neg(IrrigationNeeded \wedge CropWheat) \vee ApplyDripMethod$$

Step 2: Move negation inward

$$(\neg IrrigationNeeded \vee \neg CropWheat) \vee ApplyDripMethod$$

Step 3: Associativity of $\vee$

$$\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod$$

This is already a single disjunction (a clause).

CNF (P2):

$$(\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod)$$

3.3 CNF for P3

Original:

$$\neg ApplyDripMethod \wedge CropWheat \rightarrow CropAtRisk$$

Step 1: Eliminate implication

$$\neg(\neg ApplyDripMethod \wedge CropWheat) \vee CropAtRisk$$

Step 2: Move negation inward

$$(\neg \neg ApplyDripMethod \vee \neg CropWheat) \vee CropAtRisk$$

$$(ApplyDripMethod \vee \neg CropWheat) \vee CropAtRisk$$

Step 3: Associativity of $\vee$

$$ApplyDripMethod \vee \neg CropWheat \vee CropAtRisk$$

CNF (P3):

$$(ApplyDripMethod \vee \neg CropWheat \vee CropAtRisk)$$

3.4 CNF for facts

Facts are already atomic propositions and thus CNF unit clauses:

- **F1:** *SoilDry*
- **F2:** *CropWheat*

Complete KB in CNF:

1. $(\neg SoilDry \vee IrrigationNeeded)$
2. $(\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod)$
3. $(ApplyDripMethod \vee \neg CropWheat \vee CropAtRisk)$
4. *SoilDry*
5. *CropWheat*

4. Part (b): Resolution refutation proof of 'ApplyDripMethod'

We want to prove:

$$ApplyDripMethod$$

Using **resolution refutation**, we:

1. Convert KB to CNF (already done).
2. Negate the goal: $\neg ApplyDripMethod$.
3. Add this to the KB and derive a contradiction (empty clause $\square$).

4.1 Clauses for resolution

- C1: $\neg SoilDry \vee IrrigationNeeded$
- C2: $\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod$
- C3: $ApplyDripMethod \vee \neg CropWheat \vee CropAtRisk$
- C4: *SoilDry*
- C5: *CropWheat*
- C6 (negated goal): $\neg ApplyDripMethod$

4.2 Resolution steps

Step 1: From SoilDry to IrrigationNeeded

Resolve C1 and C4:

- C1: $\neg SoilDry \vee IrrigationNeeded$

- C4: *SoilDry*

Resolution on *SoilDry* / $\neg \text{SoilDry}$:

Result:

- R1: *IrrigationNeeded*

Step 2: Use *IrrigationNeeded* and *CropWheat* to push towards *ApplyDripMethod*

Resolve R1 with C2 and C5.

First, resolve R1 with C2:

- R1: *IrrigationNeeded*
- C2: $\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}$

Resolution on *IrrigationNeeded* / $\neg \text{IrrigationNeeded}$:

Result:

- R2: $\neg \text{CropWheat} \vee \text{ApplyDripMethod}$

Now resolve R2 with C5:

- R2: $\neg \text{CropWheat} \vee \text{ApplyDripMethod}$
- C5: *CropWheat*

Resolution on *CropWheat* / $\neg \text{CropWheat}$:

Result:

- R3: *ApplyDripMethod*

Step 3: Resolve with negated goal

Resolve R3 with C6:

- R3: *ApplyDripMethod*
- C6: $\neg \text{ApplyDripMethod}$

Resolution on *ApplyDripMethod* / $\neg \text{ApplyDripMethod}$:

Result:

- R4: $\square$ (empty clause)

The empty clause indicates a contradiction, so the assumption $\neg \text{ApplyDripMethod}$ is unsatisfiable with the KB. Therefore:

$$\boxed{KB \models \text{ApplyDripMethod}}$$

4.3 Resolution proof tree (textual)

- Root contradiction: $\square$

- Derived from:
 - **ApplyDripMethod** (R3)
 - **¬ApplyDripMethod** (C6)
- **ApplyDripMethod** (R3) derived from:
 - **¬CropWheat $\vee$ ApplyDripMethod** (R2)
 - **CropWheat** (C5)
- **¬CropWheat $\vee$ ApplyDripMethod** (R2) derived from:
 - **IrrigationNeeded** (R1)
 - **¬IrrigationNeeded $\vee$ ¬CropWheat $\vee$ ApplyDripMethod** (C2)
- **IrrigationNeeded** (R1) derived from:
 - **SoilDry** (C4)
 - **¬SoilDry $\vee$ IrrigationNeeded** (C1)

This tree shows the logical dependency chain from facts and rules to the contradiction, confirming the theorem.

5. Part (c): Effect of removing 'SoilDry' from the KB

Now we **remove the fact**:

- *SoilDry*

The updated KB in CNF:

1. $(\neg \text{SoilDry} \vee \text{IrrigationNeeded})$
2. $(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$
3. $(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk})$
4. *CropWheat*

No explicit information now forces *SoilDry* to be true.

5.1 Step-by-step reasoning changes

Previously, we had:

- *SoilDry*
- From C1: $\text{SoilDry} \Rightarrow \text{IrrigationNeeded}$
- Then $\text{IrrigationNeeded} \wedge \text{CropWheat} \Rightarrow \text{ApplyDripMethod}$
- So *ApplyDripMethod* was **entailed**.

Now, without *SoilDry*:

- C1: $\neg \text{SoilDry} \vee \text{IrrigationNeeded}$ This does **not** give *IrrigationNeeded* directly; it only says:
 - If *SoilDry* is true, then *IrrigationNeeded* must be true.
 - But if *SoilDry* is false, the clause is still satisfied without *IrrigationNeeded*.

Since we have **no fact** about *SoilDry*, we cannot derive *IrrigationNeeded*.

Without *IrrigationNeeded*:

- C2: $\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}$ With *CropWheat* true, this becomes:
 - $\neg \text{IrrigationNeeded} \vee \text{ApplyDripMethod}$

This clause is satisfiable in multiple ways:

- If *IrrigationNeeded* is false, *ApplyDripMethod* need not be true.
- If *IrrigationNeeded* is true, *ApplyDripMethod* must be true.

But the KB does **not** force *IrrigationNeeded* to be true or false; hence *ApplyDripMethod* is **not logically entailed**.

5.2 Does 'CropAtRisk' still follow?

Recall P3 in CNF:

- C3: $\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk}$

With *CropWheat* true, this simplifies to:

- $\text{ApplyDripMethod} \vee \text{CropAtRisk}$

This means:

- If *ApplyDripMethod* is **false**, then *CropAtRisk* must be **true**.
- If *ApplyDripMethod* is **true**, *CropAtRisk* may or may not be true (depending on other info).

However, we **do not know** from the KB whether *ApplyDripMethod* is true or false, because *IrrigationNeeded* is undetermined. Therefore:

- The KB does **not** force *CropAtRisk* to be true.
- Both models are possible:
 - Model 1: *ApplyDripMethod* = true, *CropAtRisk* = false (satisfies C3).
 - Model 2: *ApplyDripMethod* = false, *CropAtRisk* = true (also satisfies C3).

So **CropAtRisk** is **not logically entailed** after removing *SoilDry*.

5.3 Logical conclusion

- With **SoilDry present**, the chain is:

$$\text{SoilDry} \Rightarrow \text{IrrigationNeeded} \Rightarrow \text{ApplyDripMethod}$$

Hence ApplyDripMethod is entailed, and CropAtRisk is **not** forced (because drip method is applied).

- With **SoilDry removed**, IrrigationNeeded is not derivable, ApplyDripMethod is not entailed, and CropAtRisk becomes **possible but not guaranteed**. The system cannot conclusively say the crop is at risk.

Thus, removing SoilDry weakens the KB and breaks the deterministic reasoning chain, turning both ApplyDripMethod and CropAtRisk into **non-entailed** (uncertain) propositions.

6. Final conclusion

This case study shows how an agricultural expert system uses propositional logic to reason about irrigation and crop risk. CNF conversion and resolution refutation confirm that ApplyDripMethod is a logical consequence of the full knowledge base. However, the removal of the SoilDry fact disrupts the inference chain, making both irrigation decisions and risk assessment logically uncertain. This highlights the importance of complete and accurate sensor data (like soil moisture) for reliable expert reasoning in agricultural domains.

Case Study 4 — Wumpus World Knowledge Agent

1. Introduction

This case study analyzes how a logical AI agent in the Wumpus World uses propositional rules and percepts (Stench, Breeze, Glitter) to build a belief state, infer possible hazard locations, and decide whether a specific path is safe to reach the gold. The focus is on Modus Ponens and Forward Chaining as reasoning mechanisms.

2. Knowledge base and percepts

2.1 Rules

1. **Stench rule:** $\text{Stench}[1,2] \rightarrow \text{Wumpus is adjacent to cell } [1,2]$
2. **Breeze rule:** $\text{Breeze}[1,1] \rightarrow \text{Pit is adjacent to cell } [1,1]$

3. **Glitter rule:** $\text{Glitter}[x,y] \rightarrow \text{Gold is at } [x,y]$
4. **Safety rule:** $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

2.2 Percepts

- Stench at [1,2]
- Breeze at [1,1]
- Glitter at [2,2]

Formally:

- $\text{Stench}[1,2]$
- $\text{Breeze}[1,1]$
- $\text{Glitter}[2,2]$

3. Part (a): Belief state and Modus Ponens derivations

Modus Ponens: If $P \rightarrow Q$ and P are both true, then infer Q .

3.1 From Stench[1,2]

Rule 1: $\text{Stench}[1,2] \rightarrow \text{Wumpus is adjacent to } [1,2]$

Given percept: Stench[1,2] is true.

Conclusion 1: Wumpus is adjacent to [1,2].

In a typical Wumpus grid, “adjacent” means one of the neighboring cells (up, down, left, right). Assuming standard coordinates, possible adjacent cells to [1,2] are:

- [1,1]
- [1,3]
- [2,2]

So the belief state includes:

- $\text{Wumpus} \in \{ [1,1], [1,3], [2,2] \}$ (possible locations)

3.2 From Breeze[1,1]

Rule 2: $\text{Breeze}[1,1] \rightarrow \text{Pit is adjacent to } [1,1]$

Given percept: Breeze[1,1] is true.

Conclusion 2: Pit is adjacent to [1,1].

Adjacent cells to [1,1]:

- [1,2]
- [2,1]

So the belief state includes:

- $\text{Pit} \in \{ [1,2], [2,1] \}$ (possible locations)

3.3 From Glitter[2,2]

Rule 3: $\text{Glitter}[x,y] \rightarrow \text{Gold is at } [x,y]$

Given percept: Glitter[2,2].

Substitute $x = 2, y = 2$:

Conclusion 3: Gold is at [2,2].

3.4 Safety rule (Rule 4)

Rule 4: $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

At this stage, we **do not yet know** any cell that is guaranteed free of both Wumpus and Pit, so we cannot apply Modus Ponens on Rule 4 directly. Safety conclusions will depend on further reasoning in part (b) and (c).

3.5 Summary of derived belief state (from Modus Ponens)

- Wumpus is adjacent to [1,2] $\rightarrow \text{Wumpus} \in \{ [1,1], [1,3], [2,2] \}$
- Pit is adjacent to [1,1] $\rightarrow \text{Pit} \in \{ [1,2], [2,1] \}$
- Gold is at [2,2]

These form the core belief state of the agent.

4. Part (b): Forward chaining to identify possible Wumpus and Pit locations

Forward chaining: Start from known facts (percepts) and repeatedly apply rules to derive all possible consequences.

4.1 Initial facts

- F1: Stench[1,2]
- F2: Breeze[1,1]

- F3: Glitter[2,2]

4.2 Apply Rule 1 (Stench)

From F1 and Rule 1:

- Stench[1,2] $\rightarrow$ Wumpus adjacent to [1,2]

Forward chaining yields:

- WumpusAdj[1,2] (conceptually)
- Therefore, Wumpus $\in \{ [1,1], [1,3], [2,2] \}$

We do **not** yet know which of these is actual; all are possible.

4.3 Apply Rule 2 (Breeze)

From F2 and Rule 2:

- Breeze[1,1] $\rightarrow$ Pit adjacent to [1,1]

Forward chaining yields:

- PitAdj[1,1]
- Therefore, Pit $\in \{ [1,2], [2,1] \}$

Again, both are possible.

4.4 Apply Rule 3 (Glitter)

From F3 and Rule 3:

- Glitter[2,2] $\rightarrow$ Gold at [2,2]

Forward chaining yields:

- Gold[2,2]

4.5 Safety rule (Rule 4) and constraints

Rule 4: $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

To use forward chaining here, we need explicit facts of the form $\neg \text{Wumpus}[x,y]$ and $\neg \text{Pit}[x,y]$. The current KB does not provide such negative information directly.

Therefore:

- We **cannot** yet assert Safe[1,2] or Safe[2,2] purely from Rule 4.
- Safety must be evaluated based on possible vs. certain hazard locations.

4.6 Final forward-chaining conclusions

- **Possible Wumpus locations:** [1,1], [1,3], [2,2]
- **Possible Pit locations:** [1,2], [2,1]
- **Gold location:** [2,2]

No cell is yet guaranteed safe by Rule 4 alone.

5. Part (c): Safety evaluation of path [1,1] → [1,2] → [2,2]

We evaluate whether the agent can safely follow:

- Start: [1,1]
- Move to: [1,2]
- Then to: [2,2] (where gold is)

5.1 Cell [1,1]

We know:

- Breeze[1,1] → Pit adjacent, but not in [1,1] itself (pits cause breeze in neighboring cells).
- No rule indicates Wumpus at [1,1] directly; only that Wumpus is adjacent to [1,2], which includes [1,1] as a possibility.

So:

- Pit is **not** implied to be at [1,1].
- Wumpus **may** be at [1,1] (from adjacency to [1,2]).

Without additional rules (e.g., starting cell is safe), [1,1] is **not guaranteed safe** by the given KB alone. In standard Wumpus World, the starting cell is assumed safe, but that assumption is not explicitly encoded here.

5.2 Cell [1,2]

From part (b):

- Pit ∈ { [1,2], [2,1] }
- Wumpus ∈ { [1,1], [1,3], [2,2] }

So:

- [1,2] is a **possible Pit location**.
- [1,2] is **not** listed as a possible Wumpus location (only adjacent to [1,2], not at [1,2] itself, as per the given rule wording).

Thus:

- Risk at [1,2] comes from Pit, not Wumpus.
- Since Pit[1,2] is possible and not ruled out, [1,2] is **not guaranteed safe**.

Rule 4 would require:

- $\neg \text{Wumpus}[1,2] \wedge \neg \text{Pit}[1,2]$ to infer Safe[1,2].
- We do **not** have $\neg \text{Pit}[1,2]$; in fact, Pit[1,2] is a candidate.

Therefore, logically:

- Safe[1,2] cannot be inferred.
- Path step [1,1] $\rightarrow$ [1,2] is **potentially unsafe**.

5.3 Cell [2,2]

We know:

- Gold[2,2] (from Glitter).
- $\text{Wumpus} \in \{ [1,1], [1,3], [2,2] \} \rightarrow \text{Wumpus may be at } [2,2]$.
- Pit locations are $\{ [1,2], [2,1] \}$, so Pit[2,2] is **not** currently a candidate.

Thus:

- [2,2] may contain the Wumpus.
- Again, Rule 4 needs $\neg \text{Wumpus}[2,2] \wedge \neg \text{Pit}[2,2]$ to infer Safe[2,2], which we do not have.

So:

- Safe[2,2] cannot be inferred.
- The final step [1,2] $\rightarrow$ [2,2] is also **potentially unsafe**.

5.4 Overall safety assessment of the path

Using the derived knowledge:

- **[1,1]:** Possibly Wumpus (adjacent to [1,2]), but not explicitly ruled out; no Pit indicated.
- **[1,2]:** Possible Pit location; not safe by Rule 4.
- **[2,2]:** Contains Gold, but also a possible Wumpus location.

From a strict logical standpoint (with only the given rules):

- The agent **cannot prove** that any of these cells are safe.
- Therefore, the path [1,1] $\rightarrow$ [1,2] $\rightarrow$ [2,2] is **not logically guaranteed safe**.

In other words:

$$KB \models \neg Safe[1, 2] \text{ and } KB \models \neg Safe[2, 2]$$

So the agent, being cautious and logic-driven, should **not** treat this path as safe based solely on the current KB.

6. Final conclusion

From the percepts and rules, the agent correctly infers possible locations of the Wumpus and Pit and the exact location of the gold. However, the knowledge base does not provide enough information to prove that the path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$ is safe. Both $[1,2]$ and $[2,2]$ remain potentially hazardous due to possible Pit and Wumpus presence. For the agent to safely navigate, additional rules or observations (such as confirmed safe cells or more percepts) are required to eliminate these uncertainties.